

Pandas

What is Pandas?

Pandas is a powerful Python library used for working with datasets. It provides tools for data analysis, data cleaning, data exploration, and data manipulation. It is especially useful for structured data such as tables, CSV files, Excel files, and SQL-style data.

The two main Pandas data structures are:

- Series — one-dimensional data structure, similar to a column.
- DataFrame — two-dimensional data structure, similar to a table or spreadsheet.

Example

```
import pandas as pd

data = {"Name": ["John", "Anna", "Peter"],
        "Age": [28, 24, 35]}

df = pd.DataFrame(data)
print(df)
```

Output:

```
   Name  Age
0  John   28
1  Anna   24
2  Peter  35
```

Why Use Pandas

Pandas makes it easier to analyze, clean, explore, and manipulate structured data. It is also useful for data wrangling, where raw data is transformed into a more useful format.

- Analyze datasets and calculate statistics.
- Clean missing or unwanted data.
- Select and filter required rows and columns.
- Group data and calculate summaries.
- Combine multiple datasets.
- Read and write common data formats such as CSV and JSON.

Installation and Import

Pandas can be installed using pip and imported using the conventional pd alias.

Example

```
# Command Prompt / Terminal
pip install pandas

# Python
import pandas as pd
print(pd.__version__)
```

Output:

Example output:

2.x.x

(The exact version depends on the installed Pandas version.)

Pandas Series

A Series is a one-dimensional array-like Pandas object that can hold data and an associated index. It can be thought of as a single column of a table.

Example

```
import pandas as pd

s = pd.Series([23, 24, 25, 12, 32])
print(s)
Output:
0    23
1    24
2    25
3    12
4    32
dtype: int64
```

Custom Index

A custom index lets you assign your own labels instead of the default numeric indexes.

Example

```
s = pd.Series([23, 45, 67], index=["a", "b", "c"])
print(s)
print(s["b"])
Output:
a    23
b    45
c    67
dtype: int64
45
```

Series from a Dictionary

A dictionary can be used to create a Series, where dictionary keys become the index labels.

Example

```
data = {"day1": 420, "day2": 380, "day3": 390}
s = pd.Series(data)
print(s)
Output:
day1    420
day2    380
day3    390
dtype: int64
```

DataFrame

A DataFrame is a two-dimensional data structure with rows and columns. It is similar to a spreadsheet or a database table.

Example

```
data = {
    "Name": ["John", "Anna", "Peter"],
    "Age": [28, 24, 35]
}

df = pd.DataFrame(data)
print(df)
Output:
   Name  Age
0  John   28
```

```
1 Anna 24
2 Peter 35
```

Creating DataFrames

DataFrames can be created from dictionaries, lists, nested lists, Series, and other tabular structures.

Example

```
df = pd.DataFrame(
    [[2, 3, 4], [4, 5, 6], [1, 2, 3]],
    columns=["a", "b", "c"],
    index=["x", "y", "z"]
)
print(df)
```

Output:

```
   a  b  c
x  2  3  4
y  4  5  6
z  1  2  3
```

Converting a Series to a DataFrame

Example

```
s = pd.Series([10, 20, 30], name="Marks")
df = pd.DataFrame(s)
print(df)
```

Output:

```
   Marks
0     10
1     20
2     30
```

Reading Data from Files

Pandas provides functions to load data from common file and database formats.

Function	Purpose
<code>pd.read_csv()</code>	Read CSV files
<code>pd.read_excel()</code>	Read Excel files
<code>pd.read_json()</code>	Read JSON files
<code>pd.read_sql()</code>	Read SQL database data
<code>pd.read_html()</code>	Read tables from HTML

Example

```
df = pd.read_csv("data.csv")
print(df.head())
```

Output:

Output:

The first 5 rows of data.csv are displayed.

Writing Data to Files

Pandas can save a DataFrame to several common formats.

Example

```
df.to_csv("output.csv", index=False)
```

Output:

Output:

Creates/saves the DataFrame as output.csv without writing the DataFrame index.

Other commonly used methods include `to_excel()`, `to_json()`, and `to_sql()`.

Viewing and Understanding Data

These methods help inspect a DataFrame before performing analysis.

head() — Displays the first 5 rows by default.

tail() — Displays the last 5 rows by default.

info() — Displays information about columns, data types, non-null values, and memory usage.

describe() — Displays statistical summary information.

shape — Returns (number of rows, number of columns).

columns — Returns column names.

index — Returns row index information.

Example

```
df = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Age": [20, 25, 30]
})
```

```
print(df.head(2))
print(df.shape)
print(df.columns)
```

Output:

```
   Name  Age
0    A   20
1    B   25
(3, 2)
Index(['Name', 'Age'], dtype='object')
```

Selecting Columns

Column selection is used to retrieve one or more columns from a DataFrame.

Example

```
df = pd.DataFrame({
    "Name": ["John", "Anna", "Peter"],
    "Age": [28, 24, 35],
    "City": ["Delhi", "Pune", "Hyderabad"]
})
```

```
print(df["Name"])
print(df[["Name", "Age"]])
```

Output:

```
0    John
1    Anna
2    Peter
Name: Name, dtype: object
```

```
   Name  Age
0  John  28
1  Anna  24
2  Peter  35
```

Selecting Rows with loc and iloc

loc selects data using row/column labels, while iloc selects data using integer positions.

Example

```
df = pd.DataFrame({
    "Name": ["John", "Anna", "Peter"],
    "Age": [28, 24, 35]
})
```

```
print(df.loc[0])
```

```
print(df.iloc[1])
print(df.loc[0:1, ["Name", "Age"]])
print(df.iloc[0:2, 0:2])
```

Output:

```
Name    John
Age      28
Name: 0, dtype: object
```

```
Name    Anna
Age      24
Name: 1, dtype: object
```

```
      Name Age
0  John   28
1  Anna   24
```

```
      Name Age
0  John   28
1  Anna   24
```

Filtering Data

Filtering selects only rows that satisfy a condition.

Example

```
df = pd.DataFrame({
    "name": ["rk", "rama", "krishna"],
    "id": [102, 103, 104],
    "salary": [15000, 20000, 25000]
})
```

```
result = df[df["salary"] >= 20000]
print(result)
print(result[["id", "salary"]])
```

Output:

```
      name  id  salary
1    rama  103  20000
2 krishna  104  25000
```

```
      id  salary
1  103  20000
2  104  25000
```

Adding and Modifying Columns

A new column can be created by assigning a list or a calculated Series. Existing columns can also be modified using expressions.

Example

```
df = pd.DataFrame({
    "Name": ["John", "Anna", "Peter"],
    "Age": [28, 24, 35]
})
```

```
df["Salary"] = [50000, 60000, 70000]
df["Age"] = df["Age"] + 1
print(df)
```

Output:

```
      Name Age Salary
0   John  29  50000
1   Anna  25  60000
2  Peter  36  70000
```

Removing Rows and Columns

`drop()` removes rows or columns. The `axis` argument specifies whether the operation is across rows (`axis=0`) or columns (`axis=1`).

Example

```
df = pd.DataFrame({
    "Name": ["John", "Anna", "Peter"],
    "Age": [28, 24, 35],
    "Salary": [50000, 60000, 70000]
})

print(df.drop("Age", axis=1))
print(df.drop(0, axis=0))
```

Output:

```
   Name  Salary
0  John  50000
1  Anna  60000
2  Peter  70000
```

```
   Name Age  Salary
1  Anna  24  60000
2  Peter  35  70000
```

Inserting Columns

`insert()` adds a new column at a specified column position.

Example

```
df = pd.DataFrame({
    "a": [2, 4, 1],
    "c": [4, 6, 3]
})

df.insert(1, "b", [3, 5, 2])
print(df)
```

Output:

```
   a  b  c
0  2  3  4
1  4  5  6
2  1  2  3
```

Missing Data

Missing data represents values that are not available. Pandas provides methods to detect, remove, and replace missing values.

isnull() — Checks for missing values.

notnull() — Checks for values that are not missing.

dropna() — Removes rows/columns containing missing values.

fillna() — Replaces missing values with a specified value.

Example

```
df = pd.DataFrame({
    "Name": ["A", None, "C"],
    "Age": [20, 25, None]
})

print(df.isnull())
print(df.isnull().sum())
print(df.fillna(0))
print(df.dropna())
```

Output:

```

      Name  Age
0  False  False
1   True  False
2  False   True

```

```

Name    1
Age     1
dtype: int64

```

```

      Name  Age
0   A  20.0
1   0  25.0
2   C   0.0

```

```

      Name  Age
0   A  20.0

```

Sorting Data

Sorting arranges rows according to column values or index values.

Example

```

df = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Age": [35, 20, 28]
})

print(df.sort_values("Age"))
print(df.sort_values("Age", ascending=False))
print(df.sort_index())

```

Output:

```

      Name  Age
1   B   20
2   C   28
0   A   35

```

```

      Name  Age
0   A   35
2   C   28
1   B   20

```

```

      Name  Age
0   A   35
1   B   20
2   C   28

```

Aggregation Functions

Aggregation functions summarize data into values such as totals, averages, minimums, maximums, counts, and measures of variation.

Example

```

df = pd.DataFrame({
    "Age": [20, 30, 40, 50]
})

print("sum:", df["Age"].sum())
print("mean:", df["Age"].mean())
print("median:", df["Age"].median())
print("min:", df["Age"].min())

```

```
print("max:", df["Age"].max())
print("count:", df["Age"].count())
```

Output:

```
sum: 140
mean: 35.0
median: 35.0
min: 20
max: 50
count: 4
```

Other common aggregation functions from the supplied notes: std(), var(), and mode().

GroupBy

groupby() divides data into groups based on one or more columns and allows an aggregation or other operation to be performed on each group.

Example

```
df = pd.DataFrame({
    "Department": ["IT", "IT", "HR", "HR"],
    "Salary": [50000, 70000, 40000, 60000]
})

result = df.groupby("Department")["Salary"].mean()
print(result)
```

Output:

```
Department
HR    50000.0
IT    60000.0
Name: Salary, dtype: float64
```

Common group operations include sum(), mean(), count(), max(), min(), and agg().

Apply Functions

apply() applies a function to values along a Series or DataFrame axis.

Example

```
import pandas as pd

def calc_sum(x):
    return x + 1

df = pd.DataFrame({"x": [50, 40, 30],
                  "y": [300, 1112, 42]})

print(df.apply(calc_sum))
```

Output:

```
   x  y
0  51 301
1  41 1113
2  31  43
```

Unique and Value Counts

unique() returns the distinct values in a Series. value_counts() counts how often each value occurs.

Example

```
df = pd.DataFrame({
    "Department": ["IT", "HR", "IT", "Sales", "HR"]
})

print(df["Department"].unique())
print(df["Department"].value_counts())
```

Output:

```
['IT' 'HR' 'Sales']
```

```
Department
```

```
IT      2
```

```
HR      2
```

```
Sales   1
```

```
Name: count, dtype: int64
```

Replace Values

`replace()` substitutes specified values with other values.

Example

```
df = pd.DataFrame({"Gender": ["Male", "Female", "Male"]})
```

```
df = df.replace("Male", "M")
```

```
print(df)
```

Output:

```
Gender
```

```
0      M
```

```
1  Female
```

```
2      M
```

Indexing and Resetting Index

`set_index()` makes an existing column the DataFrame index. `reset_index()` restores the index to a regular column and creates the default numeric index.

Example

```
df = pd.DataFrame({  
    "Name": ["John", "Anna"],  
    "Age": [28, 24]  
})
```

```
print(df.set_index("Name"))
```

```
print(df.set_index("Name").reset_index())
```

Output:

```
Age
```

```
Name
```

```
John  28
```

```
Anna  24
```

```
      Name  Age
```

```
0  John   28
```

```
1  Anna   24
```

Iterating Through DataFrames

`iterrows()` allows you to iterate through a DataFrame row by row. `items()` can be used to iterate through columns.

Example

```
df = pd.DataFrame({  
    "Name": ["John", "Anna"],  
    "Age": [28, 24]  
})
```

```
for index, row in df.iterrows():
```

```
    print(row["Name"])
```

Output:

John
Anna

Transpose

The `T` attribute transposes a `DataFrame`, changing rows into columns and columns into rows.

Example

```
df = pd.DataFrame({  
    "A": [1, 2],  
    "B": [3, 4]  
})
```

```
print(df.T)
```

Output:

```
0 1  
A 1 2  
B 3 4
```

Concatenating DataFrames

`concat()` combines multiple Series or DataFrames along rows or columns.

Example

```
india = pd.DataFrame({  
    "city": ["mumbai", "delhi"],  
    "temperature": [32, 45]  
})
```

```
us = pd.DataFrame({  
    "city": ["new york", "chicago"],  
    "temperature": [21, 14]  
})
```

```
result = pd.concat([india, us], ignore_index=True)  
print(result)
```

Output:

```
city temperature  
0 mumbai      32  
1 delhi       45  
2 new york    21  
3 chicago    14
```

Concat with Keys

Example

```
result = pd.concat([india, us], keys=["india", "us"])  
print(result)
```

Output:

```
city temperature  
india 0 mumbai      32  
      1 delhi       45  
us    0 new york    21  
      1 chicago    14
```

Merging DataFrames / SQL Joins

`merge()` combines DataFrames using one or more matching columns. The `how` argument controls the join type.

Example

```
df1 = pd.DataFrame({
    "name": ["Sally", "Mary", "John"],
    "age": [50, 40, 30]
})

df2 = pd.DataFrame({
    "name": ["Sally", "Peter", "Micky"],
    "age": [77, 44, 22]
})

print(pd.merge(df1, df2, on="name", how="inner"))
```

Output:

```
   name  age_x  age_y
0  Sally    50    77
```

Common join types:

- inner — only matching rows.
- outer — all rows from both DataFrames.
- left — all rows from the left DataFrame.
- right — all rows from the right DataFrame.

Example

```
result = pd.merge(df1, df2, on="name", how="outer")
print(result)
```

Output:

```
   name  age_x  age_y
0  Sally    50.0    77.0
1   Mary    40.0   NaN
2   John    30.0   NaN
3   Peter   NaN    44.0
4   Micky   NaN    22.0
```

Removing Duplicates

`drop_duplicates()` removes duplicate rows. `duplicated()` identifies duplicate rows with Boolean values.

Example

```
df = pd.DataFrame({
    "Name": ["Parker", "Smith", "William", "Parker"],
    "Age": [21, 32, 29, 21]
})

print(df.duplicated())
print(df.drop_duplicates())
```

Output:

```
0  False
1  False
2  False
3   True
dtype: bool
```

```
   Name  Age
0  Parker  21
1  Smith  32
2  William 29
```

Pivot Tables

`pivot_table()` summarizes data by grouping rows and applying an aggregation function, producing a table useful for analysis.

Example

```
df = pd.DataFrame({
    "Department": ["IT", "IT", "HR", "HR"],
    "Salary": [50000, 70000, 40000, 60000]
})

result = pd.pivot_table(
    df, values="Salary",
    index="Department",
    aggfunc="mean"
)

print(result)
```

Output:

	Salary
Department	
HR	50000.0
IT	60000.0

String Operations

The str accessor provides string operations for text stored in a Series.

Example

```
df = pd.DataFrame({
    "Name": ["Abhinav", "Pandas", "Python"]
})

print(df["Name"].str.lower())
print(df["Name"].str.upper())
print(df["Name"].str.contains("a"))
```

Output:

```
0  abhinav
1  pandas
2  python
Name: Name, dtype: object
```

```
0  ABHINAV
1  PANDAS
2  PYTHON
Name: Name, dtype: object
```

```
0    True
1    True
2   False
Name: Name, dtype: bool
```

Mathematical Operations

Pandas supports arithmetic operations directly on Series and DataFrame columns.

Example

```
df = pd.DataFrame({
    "Age": [20, 25, 30],
    "Salary": [10000, 20000, 30000]
})

df["AgePlus5"] = df["Age"] + 5
df["DoubleSalary"] = df["Salary"] * 2
print(df)
```

Output:

	Age	Salary	AgePlus5	DoubleSalary
0	20	10000	25	20000

1	25	20000	30	40000
2	30	30000	35	60000

get_dummies

get_dummies() converts categorical values into indicator/Boolean columns. This is useful when categorical data needs to be represented numerically.

Example

```
df = pd.DataFrame({
    "employmenttype": ["Service", "Business", "Student"]
})

encoded = pd.get_dummies(df["employmenttype"])
print(encoded)
```

Output:

	Business	Service	Student
0	False	True	False
1	True	False	False
2	False	False	True

Data Correlation with corr()

corr() calculates the relationship between numeric columns. The correlation coefficient ranges from -1 to 1.

Interpretation from the supplied material:

- 1 means a perfect positive relationship.
- Values close to 1 indicate a strong positive relationship.
- Values close to -1 indicate a strong negative relationship.
- Values close to 0 indicate a weak or no linear relationship.

Example

```
df = pd.DataFrame({
    "Duration": [30, 40, 50, 60],
    "Calories": [200, 300, 400, 500]
})

print(df.corr())
```

Output:

	Duration	Calories
Duration	1.0	1.0
Calories	1.0	1.0

Common Pandas Functions

The supplied notes categorize frequently used Pandas functions into creation, reading, writing, selection, cleaning, aggregation, grouping, combining, sorting, reshaping, and apply operations.

Creation — Series(), DataFrame()

Reading — read_csv(), read_excel(), read_json(), read_sql(), read_html()

Writing — to_csv(), to_excel(), to_json(), to_sql()

Selection — loc, iloc, at, iat

Cleaning — drop(), dropna(), fillna(), replace(), rename()

Aggregation — sum(), mean(), median(), mode(), count(), std(), var()

Grouping — groupby(), agg()

Combining — merge(), join(), concat()

Sorting — sort_values(), sort_index()

Reshaping — pivot(), pivot_table(), melt(), stack(), unstack()

Apply — apply(), map(), applymap()